Lab 08: Message Passing & Distributed Systems

CS426L Introduction to Parallel & Distributed Computing, Air University
Name: Waqar Ali Reg. No: 232457 Section: VII C
GitHub: https://github.com/waqar-ali0/PDC-Lab-Waqar/tree/main/PDC-Lab08

Setup: WSL Ubuntu, GCC 15.2.0, Open MPI 5.0.10, nproc = 8. Processor name: DESKTOP-IIC2T8E.

Task 1: Hello World

```
#include <stdio.h>
#include <string.h>
#include <mpi.h>

int main(int argc, char **argv)
{
    int rank, size, len;
    char name[MPI_MAX_PROCESSOR_NAME];
    char line[MPI_MAX_PROCESSOR_NAME + 64];

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(name, &len);

    // unordered: every rank prints whenever it gets here
    printf("Hello from rank %d of %d on %s\n", rank, size, name);
    fflush(stdout);

    MPI_Barrier(MPI_COMM_WORLD);

    // ordered: every rank sends its line to rank 0, rank 0 prints in order
    snprintf(line, sizeof(line), "rank %d of %d on %s", rank, size, name);
    if (rank == 0) {
        printf("--- ordered report from rank 0 ---\n");
        printf("%s\n", line);
        for (int r = 1; r < size; r++) {
            MPI_Recv(line, sizeof(line), MPI_CHAR, r, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            printf("%s\n", line);
        }
    } else {
        MPI_Send(line, strlen(line) + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

Ranks	Size reported	Order, run 1	Order, run 2	Processor name
4	4	2 0 1 3	0 1 2 3	DESKTOP-IIC2T8E
6	6	2 4 0 5 1 3	0 4 2 5 3 1	DESKTOP-IIC2T8E
8	8	7 2 3 5 6 0 4 1	7 2 5 6 0 4 1 3	DESKTOP-IIC2T8E

The unordered lines came out in a random order each time. The ordered report from rank 0 was always 0, 1, 2, ...

Questions

1. MPI_Init starts MPI and connects the process to the others. MPI_Finalize shuts it down. Calling MPI functions outside them gives an error.
2. A rank is the ID of a process, the size is how many processes there are, and MPI_COMM_WORLD is the group of all of them. Each process uses its rank and the size to work out which part of the data is its own.
3. The processes run independently, so whoever is fastest prints first. The ordered report is in order because rank 0 receives from rank 1, then 2, and so on.
4. All the ranks run on my one laptop, so the name is the same. On a cluster each node would show its own name.
5. `./hello_mpi` runs only one process (size 1). `mpirun -np 4` starts 4 processes that can talk to each other.

Task 2: Message Passing vs Shared Memory

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 10000000
#define REPS 3

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (N % size != 0) {
        if (rank == 0) printf("number of ranks must divide %d\n", N);
        MPI_Finalize();
        return 1;
    }

    int chunk = N / size;
    double *data = NULL;
    double *part = malloc(chunk * sizeof(double));

    // only rank 0 has the full array
    if (rank == 0) {
        data = malloc(N * sizeof(double));
        for (int i = 0; i < N; i++) data[i] = 1.0;
    }

    double expected = N;
    double t_coll = 0, t_p2p = 0;
    double total_coll = 0, total_p2p = 0;

    for (int rep = 0; rep < REPS; rep++) {
        // collective variant: Scatter + Reduce
        MPI_Barrier(MPI_COMM_WORLD);
        double t0 = MPI_Wtime();

        MPI_Scatter(data, chunk, MPI_DOUBLE, part, chunk, MPI_DOUBLE, 0, MPI_COMM_WORLD);
        double local = 0;
        for (int i = 0; i < chunk; i++) local += part[i];
        MPI_Reduce(&local, &total_coll, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

        t_coll += MPI_Wtime() - t0;

        // point-to-point variant: Send / Recv by hand
        MPI_Barrier(MPI_COMM_WORLD);
        t0 = MPI_Wtime();

        if (rank == 0) {
            for (int r = 1; r < size; r++)
                MPI_Send(data + r * chunk, chunk, MPI_DOUBLE, r, 0, MPI_COMM_WORLD);
            local = 0;
            for (int i = 0; i < chunk; i++) local += data[i];
            total_p2p = local;
            for (int r = 1; r < size; r++) {
                double other;
                MPI_Recv(&other, 1, MPI_DOUBLE, r, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                total_p2p += other;
```

```
            }
        } else {
            MPI_Recv(part, chunk, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            local = 0;
            for (int i = 0; i < chunk; i++) local += part[i];
            MPI_Send(&local, 1, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD);
        }

        t_p2p += MPI_Wtime() - t0;
    }

    if (rank == 0) {
        printf("ranks=%d\n", size);
        printf("collective     sum=%.0f  avg=%.2f ms  correct=%s\n",
               total_coll, t_coll / REPS * 1000, total_coll == expected ? "YES" : "NO");
        printf("point-to-point sum=%.0f  avg=%.2f ms  correct=%s\n",
               total_p2p, t_p2p / REPS * 1000, total_p2p == expected ? "YES" : "NO");
        free(data);
    }

    free(part);
    MPI_Finalize();
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 10000000
#define REPS 3

int main(void)
{
    double *data = malloc(N * sizeof(double));
    for (int i = 0; i < N; i++) data[i] = 1.0;

    double sum = 0, t = 0;

    for (int rep = 0; rep < REPS; rep++) {
        double t0 = omp_get_wtime();
        sum = 0;

        // all threads read the same array, no copying needed
        #pragma omp parallel for reduction(+:sum)
        for (int i = 0; i < N; i++) sum += data[i];

        t += omp_get_wtime() - t0;
    }

    printf("threads=%d\n", omp_get_max_threads());
    printf("reduction sum=%.0f  avg=%.2f ms  correct=%s\n",
           sum, t / REPS * 1000, sum == N ? "YES" : "NO");

    free(data);
    return 0;
}
```

Ranks / threads	MPI collective (ms)	MPI point-to-point (ms)	OpenMP reduction (ms)	Correct?
1	105.63	28.56	30.26	Y
2	63.70	21.43	10.90	Y

4	66.58	27.42	7.69	Y
8	126.11	39.82	7.15	Y

Aspect	Shared memory (OpenMP)	Message passing (MPI)
Who sees the data	All threads see the same array	Each process has its own memory
How a worker gets data	Reads the shared array directly	Rank 0 has to send it
Combining results	reduction clause	MPI_Reduce, or Send/Recv
Synchronization	Barrier at the end of the loop	Blocking send and receive
More than one machine	No	Yes
One worker fails	Whole program crashes	Whole job is aborted by default
Main cost	Starting threads	Sending the data

Comparison: In OpenMP all the threads share one memory, so they can read the array directly. In MPI each process has its own memory, so data only moves through messages. Rank 0 had to send the array to the other ranks before they could add anything, and each rank had to send its partial sum back. Because of this, MPI was slower than OpenMP on my laptop. The collective version was the slowest, because Scatter also copies rank 0's own part. Adding more ranks did not help much because sending the data costs more than adding it. MPI's advantage is that it works across many machines, which OpenMP cannot do.

Questions

1. MPI processes do not share memory, so the other ranks do not have the array until rank 0 sends it. That means MPI is a distributed-memory model.
2. OpenMP was the fastest. The MPI versions also have to copy and send the data, so adding ranks barely reduced the time.
3. A message is delivered only when the source, destination, tag and communicator match. Blocking means the call waits until the data has been received or the buffer can be reused.
4. Both versions gave the same answer. Collectives are shorter, it is harder to make mistakes with them, and MPI can do them faster internally, so they are preferred.
5. 1,000 nodes do not share memory, so OpenMP cannot run across them. MPI can, because it sends data over the network.

Task 3: Architecture Styles

```
#include <stdio.h>
#include <mpi.h>

#define TASKS 40
#define TAG_WORK 1
#define TAG_STOP 2

// the "work": square of the task number
int do_task(int t)
{
    return t * t;
}

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        // master = server, hands out tasks one by one
        int served[64] = {0};
        long total = 0;
        int next = 0, active = 0, result;
        MPI_Status st;

        // give every worker its first task
        for (int w = 1; w < size; w++) {
            if (next < TASKS) {
                MPI_Send(&next, 1, MPI_INT, w, TAG_WORK, MPI_COMM_WORLD);
                next++;
                active++;
            } else {
                MPI_Send(&next, 1, MPI_INT, w, TAG_STOP, MPI_COMM_WORLD);
            }
        }

        // whoever finishes gets the next task
        while (active > 0) {
            MPI_Recv(&result, 1, MPI_INT, MPI_ANY_SOURCE, TAG_WORK, MPI_COMM_WORLD, &st);
            int w = st.MPI_SOURCE;
            total += result;
            served[w]++;
            active--;

            if (next < TASKS) {
                MPI_Send(&next, 1, MPI_INT, w, TAG_WORK, MPI_COMM_WORLD);
                next++;
                active++;
            } else {
                MPI_Send(&next, 1, MPI_INT, w, TAG_STOP, MPI_COMM_WORLD);
            }
        }

        long expected = 0;
        for (int t = 0; t < TASKS; t++) expected += (long)t * t;

        printf("ranks=%d workers=%d tasks=%d\n", size, size - 1, TASKS);
        for (int w = 1; w < size; w++)
```

```
            printf("worker %d served %d tasks\n", w, served[w]);
        printf("total=%ld expected=%ld correct=%s\n", total, expected,
               total == expected ? "YES" : "NO");
    } else {
        // worker = client, asks for work until told to stop
        int task;
        MPI_Status st;
        while (1) {
            MPI_Recv(&task, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &st);
            if (st.MPI_TAG == TAG_STOP) break;
            int result = do_task(task);
            MPI_Send(&result, 1, MPI_INT, 0, TAG_WORK, MPI_COMM_WORLD);
        }
    }

    MPI_Finalize();
    return 0;
}
```

```
#include <stdio.h>
#include <mpi.h>

#define LAPS 1000

int main(int argc, char **argv)
{
    int rank, size, token;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // every rank only talks to its two neighbours
    int next = (rank + 1) % size;
    int prev = (rank - 1 + size) % size;

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    for (int lap = 0; lap < LAPS; lap++) {
        if (rank == 0) {
            token = 0;
            MPI_Send(&token, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
            MPI_Recv(&token, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        } else {
            MPI_Recv(&token, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            token++;   // each rank adds 1 before passing it on
            MPI_Send(&token, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
        }
    }

    double t = (MPI_Wtime() - t0) * 1000;

    if (rank == 0) {
        long messages = (long)LAPS * size;
        printf("ranks=%d laps=%d messages=%ld\n", size, LAPS, messages);
        printf("token after last lap=%d expected=%d\n", token, size - 1);
        printf("time=%.3f ms  per message=%.5f ms  correct=%s\n",
               t, t / messages, token == size - 1 ? "YES" : "NO");
    }

    MPI_Finalize();
    return 0;
}
```

Task farm (client-server)

Ranks	Workers	Tasks per worker	Correct?
4	3	13, 14, 13	Y
8	7	5, 4, 7, 5, 7, 6, 6	Y

Token ring (peer-to-peer)

Ranks	Time (ms)	Time per message (ms)	Correct?
4	2.419	0.00060	Y
8	1141.395	0.14267	Y

With 8 ranks the ring was much slower, because all 8 cores were busy and one slow rank holds up the whole ring.

Architecture styles

Style	How MPI maps onto it	Strengths	Weaknesses	Fit
Client-server	master_worker: rank 0 gives out tasks	Simple, balances load	Rank 0 is a bottleneck	3
Peer-to-peer	ring: all ranks are equal	No central bottleneck	One slow rank slows everyone	4
Layered	User → scheduler → MPI program → MPI library → network	Each layer is separate	Some overhead per layer	5

Design note

Deployment: a cluster of 8 nodes with 16 cores each, connected by a fast network. Users log in to a login node and submit jobs to a scheduler like Slurm, which starts one MPI rank per core.

Client-server: the login node and scheduler act as servers for the users. Inside the program, master_worker is client-server: rank 0 hands out tasks and the workers ask for them.

Peer-to-peer: most MPI programs run the same code on every rank, and ranks talk directly to each other, like in the ring.

Layered: user, then scheduler, then the MPI program, then the MPI library, then the network. Each layer only uses the one below it.

Recommendation: I think the layered style fits an MPI cluster best overall, with peer-to-peer used for the actual computation. The layers keep things separate, so the same program runs on a laptop or a cluster, and peer-to-peer has no single bottleneck. Client-server is still good for the scheduler and for programs with many small independent tasks.

Questions

1. master_worker is client-server. Rank 0 gives out tasks and collects results, and the other ranks do the work. Rank 0 becomes a bottleneck when there are many workers.
2. ring is peer-to-peer. Every rank does the same thing. Its speed is limited because messages go one after another.

3. User/login node → scheduler → MPI program → MPI library → network. Each layer does its own job and hides the details of the layer below it.
4. Layered with peer-to-peer compute, because it scales well and the layers keep things separate. Client-server is better for job submission or task farms.
5. If the master fails in client-server, everything stops. In peer-to-peer the other ranks get stuck waiting for the dead one. By default MPI kills the whole job when one rank fails, so the style alone does not give fault tolerance.

Output

```
### verify
$ mpicc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
$ ompi_info | grep "Open MPI:"
                Open MPI: 5.0.10
$ mpirun -np 2 hostname
DESKTOP-IIC2T8E
DESKTOP-IIC2T8E
$ nproc
8
### build
$ mpicc -O2 -std=gnu11 hello_mpi.c -o hello_mpi
  ok
$ mpicc -O2 -std=gnu11 sum_mpi.c -o sum_mpi
  ok
$ gcc -O2 -fopenmp -std=gnu11 sum_omp.c -o sum_omp
  ok
$ mpicc -O2 -std=gnu11 master_worker.c -o master_worker
  ok
$ mpicc -O2 -std=gnu11 ring.c -o ring
  ok
### task1
$ mpirun -np 4 ./hello_mpi   (run 1)
Hello from rank 2 of 4 on DESKTOP-IIC2T8E
Hello from rank 0 of 4 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 4 on DESKTOP-IIC2T8E
Hello from rank 1 of 4 on DESKTOP-IIC2T8E
Hello from rank 3 of 4 on DESKTOP-IIC2T8E
rank 1 of 4 on DESKTOP-IIC2T8E
rank 2 of 4 on DESKTOP-IIC2T8E
rank 3 of 4 on DESKTOP-IIC2T8E
$ mpirun -np 4 ./hello_mpi   (run 2)
Hello from rank 0 of 4 on DESKTOP-IIC2T8E
Hello from rank 1 of 4 on DESKTOP-IIC2T8E
Hello from rank 2 of 4 on DESKTOP-IIC2T8E
Hello from rank 3 of 4 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 4 on DESKTOP-IIC2T8E
rank 1 of 4 on DESKTOP-IIC2T8E
rank 2 of 4 on DESKTOP-IIC2T8E
rank 3 of 4 on DESKTOP-IIC2T8E
$ mpirun -np 6 ./hello_mpi   (run 1)
Hello from rank 2 of 6 on DESKTOP-IIC2T8E
Hello from rank 4 of 6 on DESKTOP-IIC2T8E
Hello from rank 0 of 6 on DESKTOP-IIC2T8E
Hello from rank 5 of 6 on DESKTOP-IIC2T8E
Hello from rank 1 of 6 on DESKTOP-IIC2T8E
Hello from rank 3 of 6 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 6 on DESKTOP-IIC2T8E
rank 1 of 6 on DESKTOP-IIC2T8E
rank 2 of 6 on DESKTOP-IIC2T8E
rank 3 of 6 on DESKTOP-IIC2T8E
rank 4 of 6 on DESKTOP-IIC2T8E
rank 5 of 6 on DESKTOP-IIC2T8E
$ mpirun -np 6 ./hello_mpi   (run 2)
Hello from rank 0 of 6 on DESKTOP-IIC2T8E
Hello from rank 4 of 6 on DESKTOP-IIC2T8E
Hello from rank 2 of 6 on DESKTOP-IIC2T8E
```

```
Hello from rank 5 of 6 on DESKTOP-IIC2T8E
Hello from rank 3 of 6 on DESKTOP-IIC2T8E
Hello from rank 1 of 6 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 6 on DESKTOP-IIC2T8E
rank 1 of 6 on DESKTOP-IIC2T8E
rank 2 of 6 on DESKTOP-IIC2T8E
rank 3 of 6 on DESKTOP-IIC2T8E
rank 4 of 6 on DESKTOP-IIC2T8E
rank 5 of 6 on DESKTOP-IIC2T8E
$ mpirun -np 8 ./hello_mpi   (run 1)
Hello from rank 7 of 8 on DESKTOP-IIC2T8E
Hello from rank 2 of 8 on DESKTOP-IIC2T8E
Hello from rank 3 of 8 on DESKTOP-IIC2T8E
Hello from rank 5 of 8 on DESKTOP-IIC2T8E
Hello from rank 6 of 8 on DESKTOP-IIC2T8E
Hello from rank 0 of 8 on DESKTOP-IIC2T8E
Hello from rank 4 of 8 on DESKTOP-IIC2T8E
Hello from rank 1 of 8 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 8 on DESKTOP-IIC2T8E
rank 1 of 8 on DESKTOP-IIC2T8E
rank 2 of 8 on DESKTOP-IIC2T8E
rank 3 of 8 on DESKTOP-IIC2T8E
rank 4 of 8 on DESKTOP-IIC2T8E
rank 5 of 8 on DESKTOP-IIC2T8E
rank 6 of 8 on DESKTOP-IIC2T8E
rank 7 of 8 on DESKTOP-IIC2T8E
$ mpirun -np 8 ./hello_mpi   (run 2)
Hello from rank 7 of 8 on DESKTOP-IIC2T8E
Hello from rank 2 of 8 on DESKTOP-IIC2T8E
Hello from rank 5 of 8 on DESKTOP-IIC2T8E
Hello from rank 6 of 8 on DESKTOP-IIC2T8E
Hello from rank 0 of 8 on DESKTOP-IIC2T8E
Hello from rank 4 of 8 on DESKTOP-IIC2T8E
Hello from rank 1 of 8 on DESKTOP-IIC2T8E
Hello from rank 3 of 8 on DESKTOP-IIC2T8E
--- ordered report from rank 0 ---
rank 0 of 8 on DESKTOP-IIC2T8E
rank 1 of 8 on DESKTOP-IIC2T8E
rank 2 of 8 on DESKTOP-IIC2T8E
rank 3 of 8 on DESKTOP-IIC2T8E
rank 4 of 8 on DESKTOP-IIC2T8E
rank 5 of 8 on DESKTOP-IIC2T8E
rank 6 of 8 on DESKTOP-IIC2T8E
rank 7 of 8 on DESKTOP-IIC2T8E
### task2
$ mpirun -np 1 ./sum_mpi
ranks=1
collective     sum=10000000  avg=105.63 ms  correct=YES
point-to-point sum=10000000  avg=28.56 ms  correct=YES
$ mpirun -np 2 ./sum_mpi
ranks=2
collective     sum=10000000  avg=63.70 ms  correct=YES
point-to-point sum=10000000  avg=21.43 ms  correct=YES
$ mpirun -np 4 ./sum_mpi
ranks=4
collective     sum=10000000  avg=66.58 ms  correct=YES
point-to-point sum=10000000  avg=27.42 ms  correct=YES
$ mpirun -np 8 ./sum_mpi
ranks=8
collective     sum=10000000  avg=126.11 ms  correct=YES
point-to-point sum=10000000  avg=39.82 ms  correct=YES
$ OMP_NUM_THREADS=1 ./sum_omp
```

```
threads=1
reduction sum=10000000  avg=30.26 ms  correct=YES
$ OMP_NUM_THREADS=2 ./sum_omp
threads=2
reduction sum=10000000  avg=10.90 ms  correct=YES
$ OMP_NUM_THREADS=4 ./sum_omp
threads=4
reduction sum=10000000  avg=7.69 ms  correct=YES
$ OMP_NUM_THREADS=8 ./sum_omp
threads=8
reduction sum=10000000  avg=7.15 ms  correct=YES
### task3
$ mpirun -np 4 ./master_worker
ranks=4 workers=3 tasks=40
worker 1 served 13 tasks
worker 2 served 14 tasks
worker 3 served 13 tasks
total=20540 expected=20540 correct=YES
$ mpirun -np 8 ./master_worker
ranks=8 workers=7 tasks=40
worker 1 served 5 tasks
worker 2 served 4 tasks
worker 3 served 7 tasks
worker 4 served 5 tasks
worker 5 served 7 tasks
worker 6 served 6 tasks
worker 7 served 6 tasks
total=20540 expected=20540 correct=YES
$ mpirun -np 4 ./ring
ranks=4 laps=1000 messages=4000
token after last lap=3 expected=3
time=2.419 ms  per message=0.00060 ms  correct=YES
$ mpirun -np 8 ./ring
ranks=8 laps=1000 messages=8000
token after last lap=7 expected=7
time=1141.395 ms  per message=0.14267 ms  correct=YES
```